

MODULE 9 · TROUBLESHOOTING

Image Pull Failures

Read the event text, not just the status

"ImagePullBackOff" doesn't say why — the event does

ErrImagePull is the first attempt failing; the kubelet then backs off and reports ImagePullBackOff while it keeps retrying. Neither status names the cause — that lives in the Pod's **Events**.

Three root causes cover almost every case: a **bad registry hostname**, a **tag that doesn't exist**, or a **missing/broken imagePullSecret**.

BY THE END YOU CAN

- Tell ErrImagePull from ImagePullBackOff
- Match an event message to its cause
- Fix each with the right pod field

Three things have to go right before the image runs



Each stage fails a different way and needs a different fix. The event message tells you which stage broke — you rarely have to guess.

What the event message is telling you

STATUS / EVENT	ROOT CAUSE	FIX
<code>ErrImagePull</code>	First pull attempt just failed	Read the Events for the reason
<code>ImagePullBackOff</code>	Kubelet retrying with growing backoff	Same cause as <code>ErrImagePull</code> — fix it, backoff won't
<code>no such host</code>	Registry hostname wrong or unreachable	Correct the hostname in <code>image:</code>
<code>manifest unknown</code> / <code>not found</code>	The image:tag doesn't exist at that registry	Correct the tag
<code>FailedToRetrieveImagePullSecret</code>	Secret missing from this namespace	Create the Secret here — Secrets don't cross namespaces
<code>denied</code> / <code>unauthorized</code>	Registry rejected the credentials	Recreate the docker-registry Secret

DIAGNOSE

One command, read top to bottom

- **describe first** — an image-pull failure never reaches logs; the container was never created
- **Read Events in order** — the oldest event is often the real cause, later ones are just retries

TRIAGE ORDER

Status class → Events text → match the snippet to a cause → apply that specific fix, not a generic retry.

```
$ kubectl describe pod NAME -n training
# Events:
#   Failed to pull image "...
#   FailedToRetrieveImagePullSecret ...

$ kubectl get pod NAME -n training
```

Two events, one root cause

A pod referencing a Secret that doesn't exist in its own namespace produces two events: the missing-secret warning fires first, then the pull itself fails on missing credentials. Secrets are namespace-scoped — one created elsewhere doesn't help here.

Fix = create/recreate the Secret in this namespace (or fix a typo'd name), then delete & reapply the pod.

```
Warning  FailedToRetrieveImagePullSecret
        secret "registry-pull-secret" not found
Warning  Failed
        Failed to pull image "repo/app:1.0":
        ... denied / unauthorized
```

THE FIX

image and imagePullSecrets can't be patched in place

Both fields are set at pod creation — there's no live edit. Every fix in this lab reduces to the same two steps: delete the pod, reapply the corrected spec.

A Deployment does this for you on every rollout; a bare Pod needs it done by hand.

```
$ kubectl delete pod NAME -n training \
    --force --grace-period=0
$ kubectl apply -f fixed-pod.yaml
$ kubectl wait --for=condition=Ready \
    pod/NAME -n training --timeout=60s
```

Where people trip

- **Assuming ImagePullBackOff means auth.** It's a generic retry state — DNS and tag errors land here too.
- **Checking logs first.** There's no container yet — everything is in Events.
- **Recreating the Secret in the wrong namespace.** Secrets never cross namespaces — only the pod's own namespace counts.
- **Editing image: and expecting it to apply.** The field is immutable — delete & reapply.

Commands to keep at hand

```
$ kubectl get pod NAME -n training           # ErrImagePull / ImagePullBackOff
$ kubectl describe pod NAME -n training | tail -20 # Events = the cause
$ kubectl get secret -n training              # does the imagePullSecret exist?
$ kubectl create secret docker-registry NAME ... # (re)create a pull secret
$ kubectl delete pod NAME --force --grace-period=0 && kubectl apply -f fixed.yaml
```

Tip: the Events section names the exact stage that broke — resolve, find, or authenticate. Fix that one thing.

RECAP

Image pull triage in one breath

- `ErrImagePull` = first failure; `ImagePullBackOff` = kubelet retrying
- Three causes: bad hostname, missing tag, missing/bad `imagePullSecret`
- Events is the only diagnostic window — there are no logs yet
- `image` / `imagePullSecrets` are immutable — delete & reapply

HANDS-ON

Now run Lab 9.2
[labs/9.2/](#)

Next: [9.3 — ConfigMap & Secret Problems](#)